



Programa Certified AI-LLM Solution Architect

Curso Diseño de Fundamentos de Arquitectura LLM

Python desde cero aplicado a modelos de lenguaje

www.bsginstitute.com

Andrés Felipe Rojas Parra

- Ingeniero electrónico con mas de 25 años de experiencia
- PGP Artificial Intelligence & Machine Learning at Texas University – Austin, TX
- Master en Inteligencia Artificial y Big Data – IEBSchool
- Estudiante de PGP en Cloud Computing at Texas University - Austin, TX
- Texas Executive Education Program – McCombs School of Business – Decision Science and AI Immersion Program
- Especialista en MLOps de la Universidad de Duke
- Estudiante de Deep Learning for Healthcare Specialization
- Director de Inteligencia Artificial e Innovacion Universidad EAN/Colombia



anrojas@universidadean.edu.co / arojaspa@outlook.com / arojaspa76



@arojaspa



<https://www.linkedin.com/in/arojaspa/>



+573005906373



NIVELACION 1a

SESION 1a

3 HORAS

Kubernetes, Docker y LLMs en la Nube

Variables, Funciones, OOP y primera llamada a un LLM



Python



dict



class



def



JSON

Agenda — Sesión 1a

Tres bloques de trabajo · 3 horas · Nivel cero en Python

01

60 min

Bloque 1 — Python Esencial

- ▶ Variables, tipos, strings, f-strings
- ▶ Funciones, *args, **kwargs, lambda
- ▶ Listas, dicts, tuplas, sets
- ▶ Clases, atributos, metodos, herencia

02

60 min

Bloque 2 — Funciones y Estructuras de Datos

- ▶ Estructuras de control: if, for, while
- ▶ Funciones de orden superior
- ▶ JSON: parsear respuestas de LLM
- ▶ Primera llamada real a Ollama

03

50 min

Bloque 3 — OOP y Primera API Call

- ▶ Comprensiones de lista
- ▶ Closures y patrones reales
- ▶ Archivos y variables de entorno
- ▶ Retry, errores, produccion



Break 10 min entre Bloque 1 y 2

¿Por que Python para IA/LLM?

Tema 1 · Python es el lenguaje #1 para inteligencia artificial

Otros lenguajes

- ✗ Java/C#: verbose, tipado estricto, compilacion lenta
- ✗ JavaScript: callbacks, async complicado para ML
- ✗ R/Matlab: nicho, curva de aprendizaje alta
- ✗ Go/Rust: rapidos pero sin librerias de IA maduras
- ✗ Otros: aprender el lenguaje + aprender IA = doble esfuerzo



Python

- ✓ Python: 3 lineas para llamar a un LLM
- ✓ Python: ecosistema IA maduro (numpy, transformers, langchain)
- ✓ Python: un script = datos + ML + API + visualizacion
- ✓ Python: OpenAI, Anthropic, HuggingFace — todos usan Python
- ✓ Python: aprende una vez, aplica en toda la cadena de IA



Variables y Tipos de Datos en Python

Tema 1 · Los bloques fundamentales — sin declarar tipos



01_variables_tipos.py

Instrucciones
para construir



Tipos basicos

Snapshot
immutable



F-strings

Proceso
ejecutándose



None / bool

ACR · ECR
Artifact Reg

Python — Variables LLM

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY ./app ./app
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0"]
```

Tipos que mas usaras

```
costo = tokens / 1_000_000 * 0.15

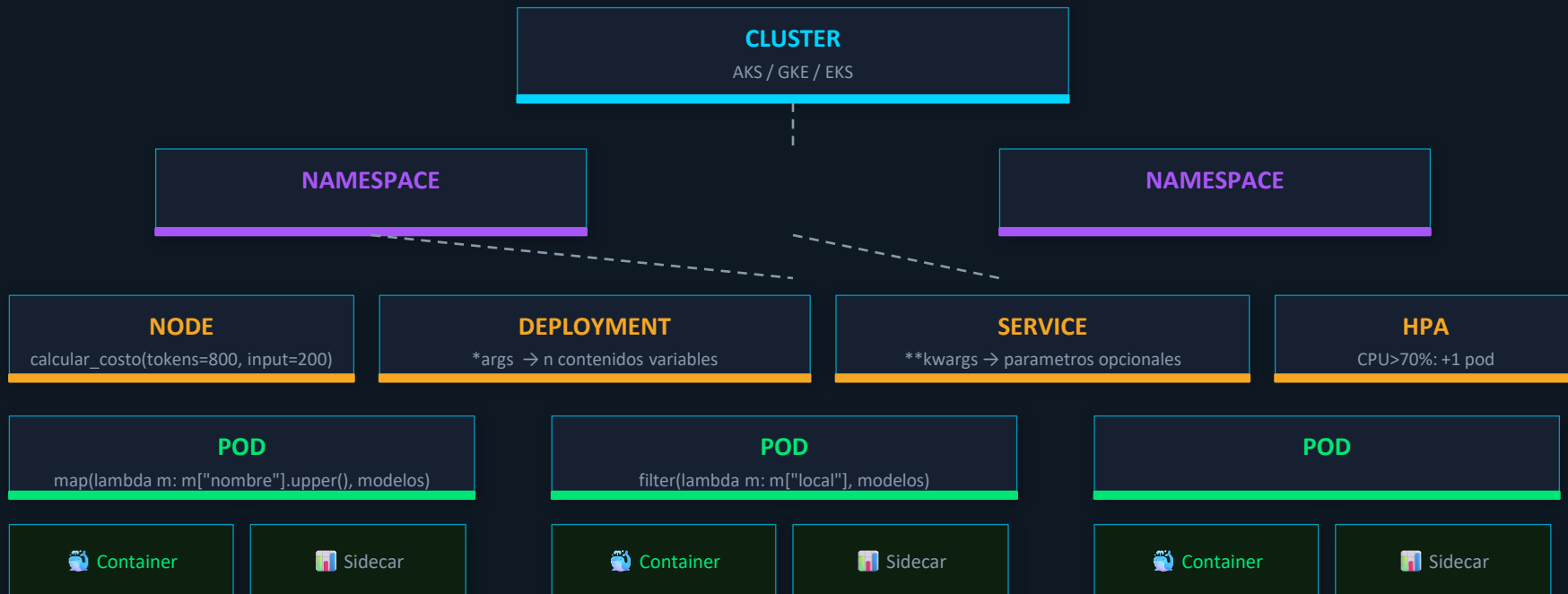
print(f"Costo: ${costo:.6f} USD")

type(nombre)  # → <class str>
```

```
# $0.000614 por
esta llamada
# Python infiere
el tipo
automáticamente.
# No se declara:
nombre: str =
"Claude"
```

Funciones — def, Args, Lambda

Tema 1 · Reutiliza logica — el SDK de OpenAI son funciones y clases



Estructuras de Control

PRACTICA

Tema 1 · if/else, for, while — aplicados a logica real de LLM



02_estructuras_control.py

```
tokens_usados = 95_000

elif tokens_usados > limite * 0.9:

else:

    print("OK: espacio suficiente")

while intentos < MAX and not exito:

[m for m in modelos if m["local"]]

[f"${c:.4f}" for c in costos]
```

Comprensiones — la forma Pythonica:

Lista: `[m for m in modelos if m["local"]]`

Dict: `{k: v*0.92 for k, v in precios.items()}`

Set: `{m["nombre"] for m in modelos}`



Comprensiones de Lista

if

```
tokens_usados > limite: print("ERROR: aplicar RAG")
```

if

```
tokens > limite * 0.7: print("OK: monitorear")
```

for

```
i, modelo in enumerate(modelos): print(f"{i}. {modelo}")
```

for

```
espera = 2 ** intentos # backoff time.sleep(espera)
```

for

```
@app.post("/chat")
async def chat(req: ChatRequest):
    resp = await ollama_client.chat(
        model=req.model,
        messages=req.messages)
    return ChatResponse(**resp)
```

Strings y F-strings — El Corazon de los Prompts

Tema 1 · Los prompts son strings — dominar strings es dominar LLMs



Operaciones basicas

```
texto.upper() → TRANSFORMER

"Trans" in texto → True

prompt = f"Usando {modelo}"

"""

costo = (tokens / 1_000_000) * precio

print(f"Costo: ${costo:.6f} USD")

docker push \

print(f"Costo: ${costo:.6f} USD")
```



```
texto = "Transformer"
te
m
```

```
modelo = llama3.2"
```

```
=
```



Prompt multilinea (triple comilla)

```
Eres un experto en Python.
Responde en espanol."""

costo = (tokens / 1_000_000) * precio
```



F-strings: formato moderno

```
len(texto) → 11

texto.split(" ") → lista

f"Temp: {temp:.2f}" → 0.70

system_prompt = ""

"""

costo = (tokens / 1_000_000) * precio

123.dkr.ecr.../llm-gateway:v1
```



Los prompts son strings multilinea. Dominar f-strings = dominar la construccion de prompts para LLM

Bloque 2 — Funciones y Estructuras de Datos

Tema 2 · Encapsulacion y las estructuras que usa todo LLM SDK

Estructura	Uso en LLM	Ejemplo	Archivo
list	Historial de chat	<code>[[role, content], ...]</code>	Historial de chat
dict	Request/Response API	<code>{"model": "llama3.2"}</code>	
tuple	Config inmutable	<code>("localhost", 11434)</code>	
set	Capacidades de modelo	<code>{"vision", "audio"}</code>	
function	Logica reutilizable	<code>calcular_costo()</code>	
class	Cliente LLM OOP	<code>ClienteOllama()</code>	
03/04_*.py	funciones.py	listas_dicts.py	



El historial de chat de un chatbot es una list de dict — estas estructuras son el corazon de cualquier LLM app

Listas y Diccionarios — El Corazon de los LLMs

Tema 2 · El historial de chat es una lista de dicts

04_listas_diccionarios.py

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: llm-gateway
  namespace: llm-prod
spec:
  replicas: 2
  selector:
    matchLabels:
      app: llm-gateway
  template:
    spec:
      containers:
        - name: llm-gateway
          image: REGISTRY/llm-gateway:v1
          ports:
```

API Request (dict):

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
```

Tuplas — inmutables, mas rapidas:

```
endpoint = ("localhost", 11434, "http")

Config = namedtuple("Config", ["nombre", "temp"])

cfg.nombre → llama3.2
```

```
host, puerto,
proto =
endpoint #
Config
Config("llama
3.2", 0.7)
```



JSON y Archivos — El Lenguaje de las APIs

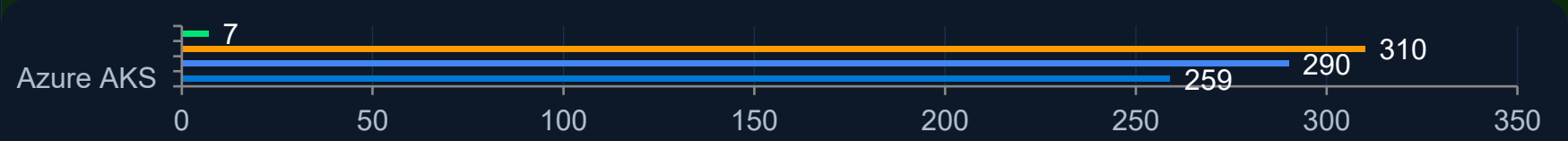
Tema 2 · `json.loads()`, `json.dumps()`, variables de entorno

json.loads() string→dict String JSON → dict Python <pre>resp = json.loads(response_str)["message"]["content"]</pre>	json.dumps() dict→string Rendimiento & Latencia <pre>json.dumps(payload, ensure_ascii=False, indent=2)</pre>	open()/with archivos Guardar/cargar historial <pre>with open("historial.json", "w") as f: json.dump(chat, f)</pre>	os.getenv() archivos Seguridad & Cumplimiento <pre>api_key = os.getenv("OPENAI_API_KEY", "no-configurada")</pre>
05_json_archivo.py Ver ejemplo 05 Usa python-dotenv + archivo .env + .gitignore	Regla de oro: Soporte & SLA <pre>OPENAI_API_KEY=sk-... en .env → os.getenv("OPENAI_API_KEY")</pre>	!	

Clases y OOP — Construye tu Cliente LLM

Tema 3 · Todo SDK de LLM es una clase — `OpenAI()`, `Anthropic()`, `Ollama()`

Concepto	Ejemplo LLM	Codigo		
class	Blueprint del objeto	class ModeloLLM:		
__init__		\$73	\$73	
atributos	self.modelo = "llama3.2"			
def chat(self, msg):	metodo	return self.llamar_api(msg)		
herencia	ModeloLocal(ModeloLLM):	metodo	ModeloLocal(ModeloLLM):	
@property	atributo con logica	conv.tokens_usados → int	return self.llamar_api(msg)	
instancia	objeto = ModeloLLM("llama3.2")	objeto.chat("Hola!")		



Primera Llamada Real a Ollama

Tema 3 · POST /api/chat — conecta todo lo aprendido

Flujo completo:

~200

ms

Primer token

~2GB

Modelo local

Gratis

Costo llamada

```
import requests
```

```
payload = {  
    "model":  
    "llama3.2  
    OLLAMA_  
    URL =  
    texto =  
    "/api/chat"  
    r.json() =  
    "message"  
    "content"
```

Codigo Python:

1. Usuario escribe
pregunta

2. Construyes payload JSON

3. POST HTTP → Ollama

4. Parseas JSON response

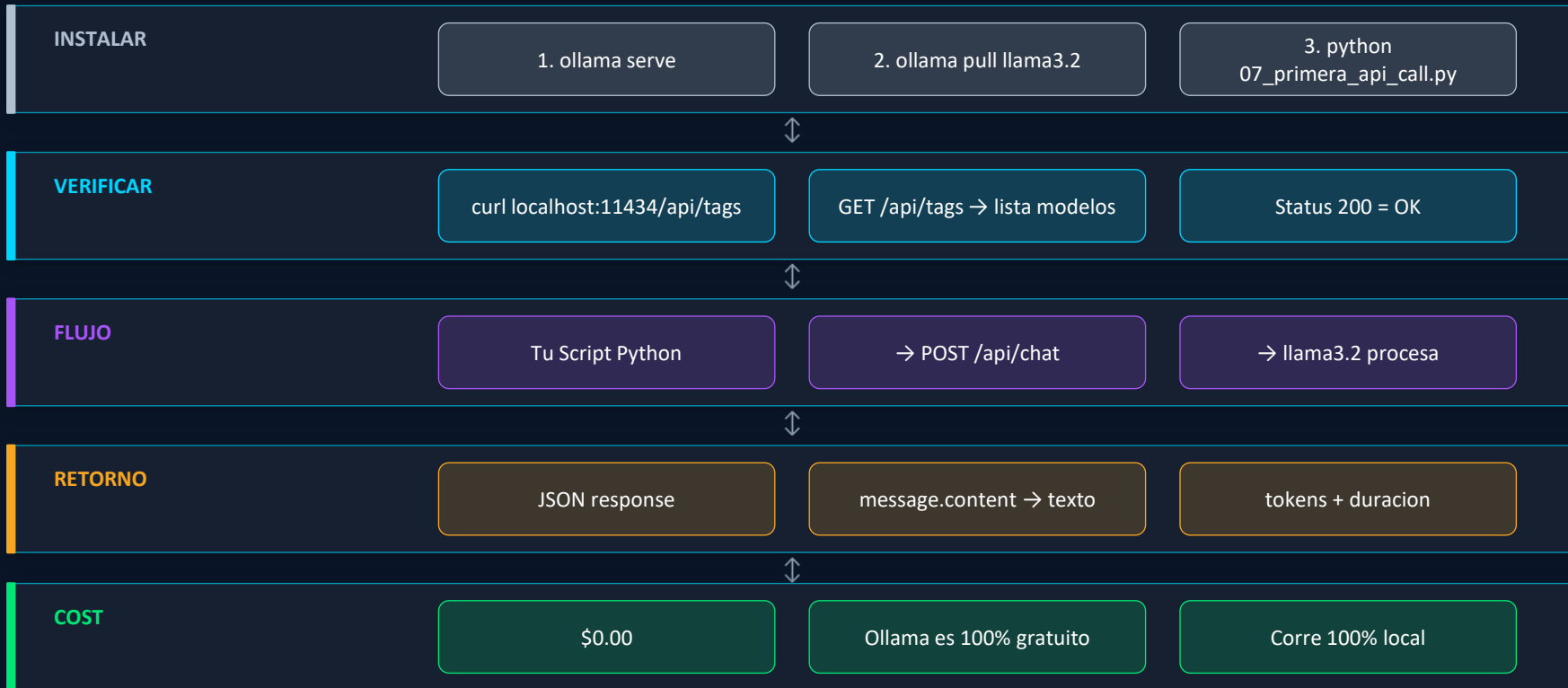
5. Muestras al usuario

"messages": historial,
"stream": False}

json=payload,
timeout=60)

Demo en Vivo — Ollama + Python

Instalacion y primera llamada en 4 pasos



Resumen — Lo que Aprendiste Hoy

Conexion entre Python y el mundo real de los LLMs

Concepto Python

str + f-strings

dict + list

funciones + closures

class + herencia

json.loads() + requests



Uso en LLM

Construir prompts dinamicos

Historial y request JSON

Encapsular logica de LLM

Cliente OOP reutilizable

Llamar y parsear APIs LLM



Siguiente sesion: Sesion 1b — APIs REST y Endpoints LLM en profundidad

Recursos y Sigüientes Pasos

TAREA

Lo que debes instalar y practicar antes de la Sesión 1b



Instalar antes de la próxima sesión:

```
pip install requests openai fastapi uvicorn
```

```
ollama serve + ollama pull llama3.2
```

Node.js 18+ para el frontend React

```
git clone <repo> + cd nivelacion
```

session_1a/ejemplos/ → 7 scripts

```
session_1a/docs/SETUP.md
```

```
requirements.txt + .env.example
```

Preguntas frecuentes:

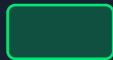
llama3.2:3b 2GB ★ Recomendado aula

mistral:7b 4GB Buenas instrucciones

phi3:mini 2.3GB Rápido y eficiente



Recursos del repositorio:



```
@app.post("/chat")
async def chat(req: ChatRequest):
    resp = await ollama_client.chat(
        model=req.model,
        messages=req.messages)
    return ChatResponse(**resp)
```



Resumen de la Sesión 1a



Python: variables, tipos, strings, f-strings



Control flow: if/elif/else, for, while, comprensiones



Funciones: def, *args, **kwargs, lambda, closures



Estructuras: list, dict, tuple, set — historial de chat



JSON y archivos: json.loads(), json.dumps(), os.getenv()



OOP: class, __init__, herencia, @property — patron del SDK

Recursos del Curso

Repositorio: github.com/bsg-institute/nivelacion-llm

Ejemplos: [session_1a/ejemplos/](#) (7 scripts listos)

Setup: [session_1a/docs/SETUP.md](#)

Ollama: `ollama serve + ollama pull llama3.2`